

Milestone 1 (just to blink)

JUST TO BLINK:

```
#include <stdint.h>
#include <msp432e401y.h>

#define CYCLE_PERIOD_MS 500
#define DELAY_VALUE (CYCLE_PERIOD_MS / 2) //since 50 percent of duty cycle

void GPIO_Init(void);
void Delay_ms(uint32_t ms);

int main(void)
{
    GPIO_Init();

    while(1) //infinite loop for LED to flash continuously
    {
        GPION-> DATA |= (1 << 2); //LED ON
        Delay_ms(DELAY_VALUE); //wait
        GPION-> DATA &= ~(1 << 2); //LED OFF
        Delay_ms(DELAY_VALUE); //wait
    }
}

void GPIO_Init(void)
{
    SYSCTL->RCGCGPIO |= (1 << 12); //enable clock for Port N
    while((SYSCTL->PRGPIO & (1<<12)) == 0); //wait for Port N to be ready
    GPION -> DIR |= (1 << 2); //set PN1 as output
```

```

    GPION -> DEN |= (1 << 2);    //enable digital function for PN2
}

```

```

void Delay_ms (uint32_t ms)
{
    uint32_t i;
    for(i=0; i<(ms*1600); i++) //recall PIOSC, how much it runs
    {
        __asm("nop");
    }
}

```

Milestone 1 (with intensity)

```

#include <stdint.h>
#include <msp432e401y.h>

```

```

#define CYCLE_PERIOD_MS 10
#define DELAY_VALUE (CYCLE_PERIOD_MS / 2) //since 50 percent of duty cycle

```

```

void GPIO_Init(void);
void Delay_ms(uint32_t ms);
void DutyCycle_Percent(uint8_t value);
void IntensitySteps(void);

```

```

int main(void)
{
    GPIO_Init();

    while(1)
    {
        IntensitySteps();
    }
}

```

```

void GPIO_Init(void)
{
    SYSCCTL->RCGCGPIO |= (1 << 12);    //enable clock for Port N
    while((SYSCCTL->PRGPIO & (1<<12)) == 0);    //wait for Port N to be ready
    GPION -> DIR |= (1 << 2);    //set PN2 as output
    GPION -> DEN |= (1 << 2);    //enable digital function for PN2
}

```

```

void Delay_ms (uint32_t ms)
{
    uint32_t i;
    for(i=0; i<(ms*1600); i++) //recall PIOSC, how much it runs
    {
        __asm("nop");
    }
}

```

```

void DutyCycle_Percent(uint8_t value)
{
    uint32_t high_time;
    uint32_t low_time;

    // Convert 0–255 to percentage of 10ms
    high_time = (value * CYCLE_PERIOD_MS) / 255;
    low_time = CYCLE_PERIOD_MS - high_time;

    GPION->DATA |= (1 << 2); // LED ON
    Delay_ms(high_time);

    GPION->DATA &= ~(1 << 2); // LED OFF
    Delay_ms(low_time);
}

```

```

void IntensitySteps(void)
{
    int i, repeat;

    // Increase 0 - 100%
    for(i = 0; i <= 250; i += 25)
    {
        for(repeat = 0; repeat < 10; repeat++)
        {

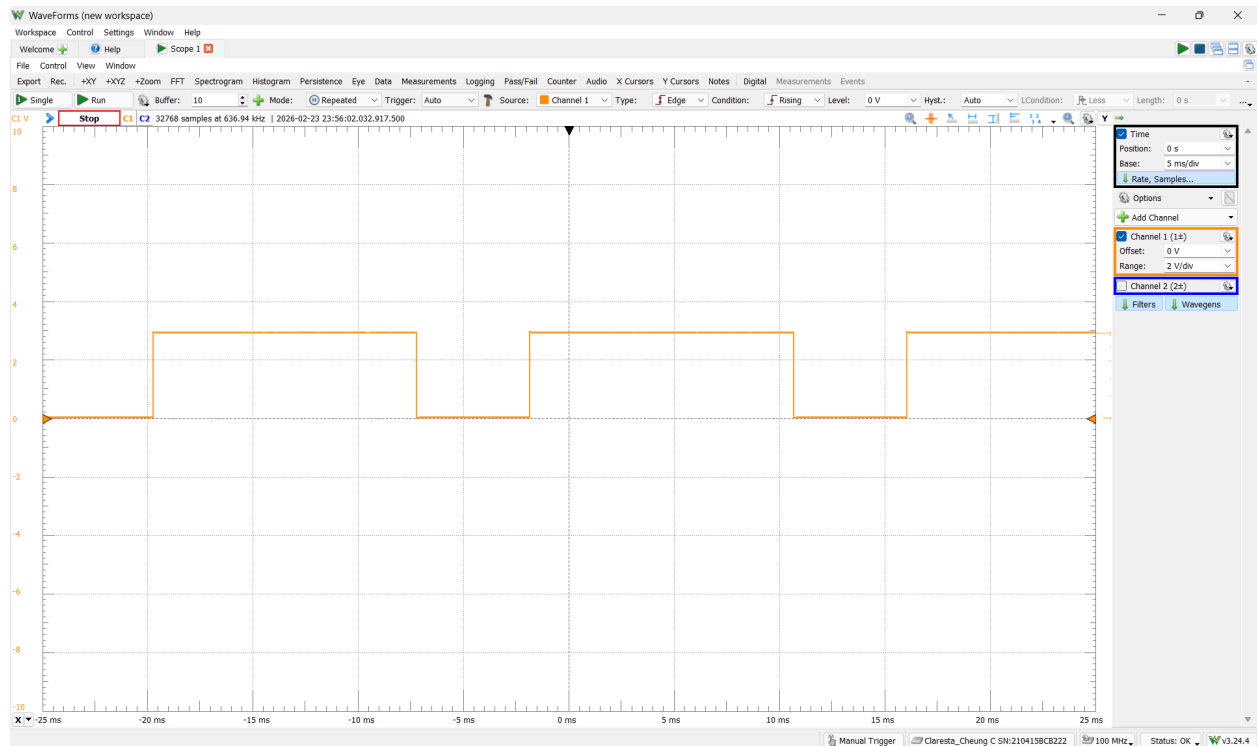
```

```

        DutyCycle_Percent(i);
    }
}

// Decrease 100 - 0%
for(i = 250; i >= 0; i -= 25)
{
    for(repeat = 0; repeat < 10; repeat++)
    {
        DutyCycle_Percent(i);
    }
}
}

```



Milestone 2

```

#include <stdint.h>
#include "tm4c1294ncpdt.h"

```

```

#include "PLL.h"
#include "SysTick.h"

// Full-step sequence for unipolar stepper
uint8_t step_sequence[4] = {0b00000011, 0b00000110, 0b00001100, 0b00001001};

void PortM_Init(void){
    SYSTCL_RCGCGPIO_R |= SYSTCL_RCGCGPIO_R11;    // Enable clock for Port M
    while((SYSTCL_PRGPIO_R & SYSTCL_PRGPIO_R11) == 0){}; // wait for clock
    GPIO_PORTM_DIR_R |= 0x0F;                    // PM0-PM3 as output
    GPIO_PORTM_AFSEL_R &= ~0x0F;                 // disable alternate function
    GPIO_PORTM_DEN_R |= 0x0F;                    // digital enable
    GPIO_PORTM_AMSEL_R &= ~0x0F;                 // disable analog
}

// Rotate stepper motor
// steps: number of steps
// delay: delay in multiples of 10ms (passed to SysTick_Wait10ms)
// direction: 1 = clockwise, -1 = counter-clockwise

void rotateStepper(int steps, int delay, int direction){
    int step_index = 0;
    for(int i = 0; i < steps; i++){
        GPIO_PORTM_DATA_R = step_sequence[step_index]; // output step
        SysTick_Wait10ms(delay);

        if(direction == 1){
            step_index = (step_index + 1) % 4;        // clockwise
        } else {
            step_index = (step_index + 3) % 4;        // counter-clockwise
        }
    }
}

int main(void){
    PLL_Init();    // Set system clock to 120 MHz
    SysTick_Init(); // Initialize SysTick
    PortM_Init();  // Initialize Port M

    int step_delay = 1;    // 10ms per step
    int steps_per_rotation = 2048; // Measure or use motor datasheet

    // Example: 2 rotations clockwise, 1 rotation counter-clockwise
    rotateStepper(2 * steps_per_rotation, step_delay, 1); // clockwise

```

```

rotateStepper(steps_per_rotation, step_delay, -1);    // counter-clockwise

while(1);
}

```

Make sure your stepper motor driver inputs connect to **PM0–PM3** on Port M as A, B, A', B'.

In1 -> PM0

In2 -> PM1

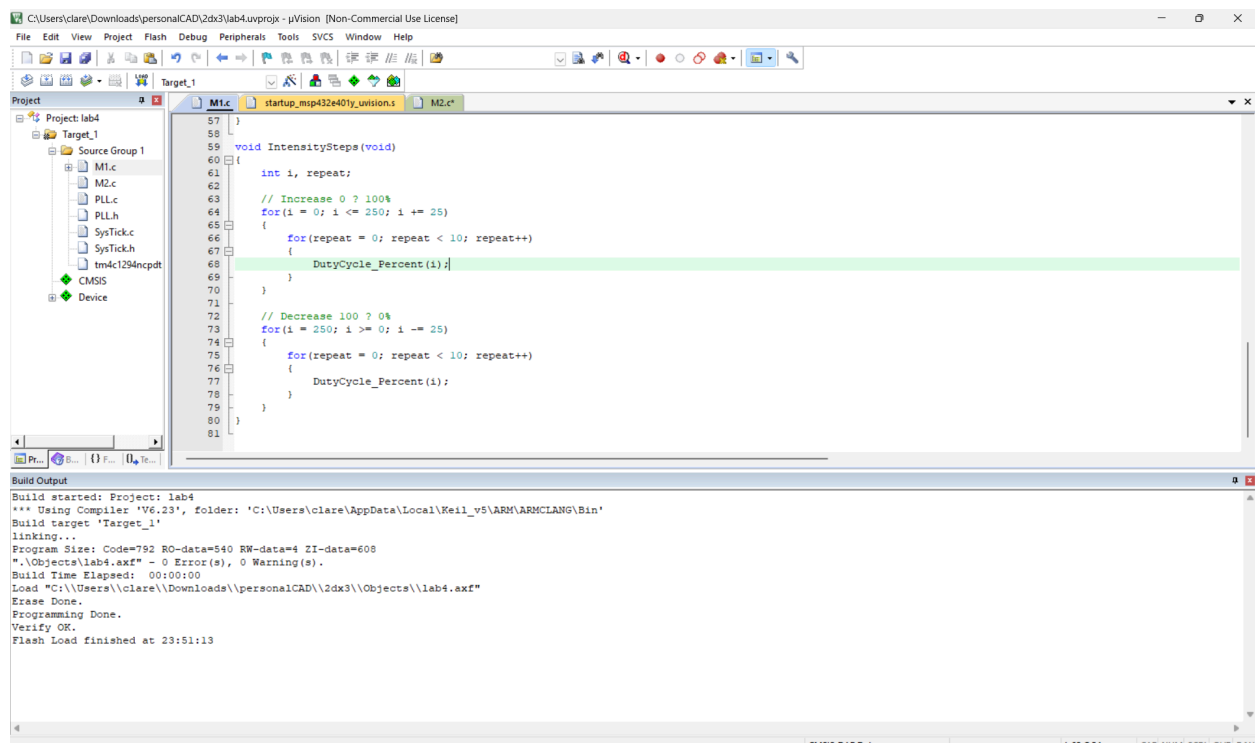
In3 -> PM2

In4 -> PM3

Ground and connect power source

- 1.) Direction controlled by incrementing or decrementing step index
- 2.) Min time delay == 200000
- 3.) 4096 steps
- 4.) demo

Make sure to include all these files:



ALTERNATE

// COMPENG 2DX3

// This program illustrates the interfacing of the Stepper Motor with the microcontroller

// Written by Ama Simons

// January 18, 2020

// Last Update by Dr. Shahrukh Athar on February 2, 2025

#include <stdint.h>

#include "tm4c1294ncpdt.h"

#include "PLL.h"

#include "SysTick.h"

void PortM_Init(void){

 //Use PortM pins (PM0-PM3) for output

 SYSCTL_RCGCGPIO_R |= SYSCTL_RCGCGPIO_R11; // activate clock for

Port M

 while((SYSCTL_PRGPIO_R&SYSCTL_PRGPIO_R11) == 0){}; // allow time for clock
to stabilize

 GPIO_PORTM_DIR_R |= 0x0F; // configure Port M pins (PM0-PM3)

as output

 GPIO_PORTM_AFSEL_R &= ~0x0F; // disable alt funct on

Port M pins (PM0-PM3)

 GPIO_PORTM_DEN_R |= 0x0F; // enable digital I/O on

Port M pins (PM0-PM3)

// configure Port M as GPIO

 GPIO_PORTM_AMSEL_R &= ~0x0F; // disable analog

functionality on Port M pins (PM0-PM3)

 return;

}

void spinCC(){

// Complete function spin to

implement the Full-step Stepping Method

 uint32_t delay = 200000;

// Does your motor spin

clockwise or counter-clockwise?

 for(int i=0; i< 1024; i++){

// What should the upper-bound of i

be for one complete rotation of the motor shaft?

 GPIO_PORTM_DATA_R = 0b00000011;

```

        SysTick_Wait (delay);                // What if we want to reduce the delay
between steps to be less than 10 ms?
        GPIO_PORTM_DATA_R = 0b00000110;      // Complete the
missing code.
        SysTick_Wait (delay);
        GPIO_PORTM_DATA_R = 0b00001100;      // Complete the
missing code.
        SysTick_Wait (delay);
        GPIO_PORTM_DATA_R = 0b00001001;      // Complete the
missing code.
        SysTick_Wait (delay);
    }
}

void spinCW(){                                // Complete function spin to
implement the Full-step Stepping Method
    uint32_t delay = 200000;                  // Does your motor spin
clockwise or counter-clockwise?

    for(int i=0; i<512; i++){                  // What should the upper-bound of i
be for one complete rotation of the motor shaft?
        GPIO_PORTM_DATA_R = 0b00001001;
        SysTick_Wait (delay);                // What if we want to reduce the delay
between steps to be less than 10 ms?
        GPIO_PORTM_DATA_R = 0b00001100;      // Complete the
missing code.
        SysTick_Wait (delay);
        GPIO_PORTM_DATA_R = 0b00000110;      // Complete the
missing code.
        SysTick_Wait (delay);
        GPIO_PORTM_DATA_R = 0b00000011;      // Complete the
missing code.
        SysTick_Wait (delay);
    }
}

int main(void){
    PLL_Init();                               // Default Set System Clock to
120MHz
    SysTick_Init();                           // Initialize SysTick configuration
    PortM_Init();                             // Initialize Port M
    //spinCC();                               // Call function spin
    spinCW();
    return 0;
}

```



```
}
```

Bonus

```
#include <stdint.h>
#include <msp432e401y.h>

#define CYCLE_PERIOD_MS 10 // Total PWM period in ms

// Pin mapping
#define RED_PIN 2
#define GREEN_PIN 3
#define BLUE_PIN 4

// Function prototypes
void GPIO_Init(void);
void Delay_ms(uint32_t ms);
void DutyCycleRGB(uint8_t red, uint8_t green, uint8_t blue);

int main(void)
{
    GPIO_Init();

    // Demo loop: cycle through colors
    while(1)
    {
        // Red only
        for(uint8_t i = 0; i <= 255; i += 5)
            DutyCycleRGB(i, 0, 0);

        // Green only
        for(uint8_t i = 0; i <= 255; i += 5)
            DutyCycleRGB(0, i, 0);

        // Blue only
        for(uint8_t i = 0; i <= 255; i += 5)
            DutyCycleRGB(0, 0, i);

        // Mix colors (purple/pink)
        for(uint8_t i = 0; i <= 255; i += 5)
            DutyCycleRGB(i, 0, 255 - i);
    }
}
```

```

    // Mix colors (teal/cyan)
    for(uint8_t i = 0; i <= 255; i += 5)
        DutyCycleRGB(0, i, 255 - i);

    // Mix colors (white fade)
    for(uint8_t i = 0; i <= 255; i += 5)
        DutyCycleRGB(i, i, i);
}
}

// Initialize PN2-PN4 as outputs
void GPIO_Init(void)
{
    SYSCTL->RCGCGPIO |= (1 << 12);          // enable clock for Port N
    while((SYSCTL->PRGPIO & (1 << 12)) == 0); // wait until ready

    GPION->DIR |= (1 << RED_PIN) | (1 << GREEN_PIN) | (1 << BLUE_PIN); // output
    GPION->DEN |= (1 << RED_PIN) | (1 << GREEN_PIN) | (1 << BLUE_PIN); // digital enable
}

// Approximate delay in ms
void Delay_ms(uint32_t ms)
{
    uint32_t i;
    for(i = 0; i < (ms * 1600); i++)
        __asm("nop");
}

// Software PWM for RGB LED
// Inputs 0–255 for each color
void DutyCycleRGB(uint8_t red, uint8_t green, uint8_t blue)
{
    uint32_t high_r = (red * CYCLE_PERIOD_MS) / 255;
    uint32_t high_g = (green * CYCLE_PERIOD_MS) / 255;
    uint32_t high_b = (blue * CYCLE_PERIOD_MS) / 255;

    // Turn on LEDs for their high portion
    GPION->DATA |= ( (high_r ? (1<<RED_PIN) : 0) |
                    (high_g ? (1<<GREEN_PIN) : 0) |
                    (high_b ? (1<<BLUE_PIN) : 0) );

    Delay_ms(1); // minimal high time (can tune if needed)

    // Turn off all LEDs

```

```
GPION->DATA &= ~( (1<<RED_PIN) | (1<<GREEN_PIN) | (1<<BLUE_PIN) );
```

```
// Delay for remaining portion of PWM cycle
```

```
Delay_ms(CYCLE_PERIOD_MS - 1);
```

```
}
```